

Constrained Creativity for SysOps Agents

Sayan Sinha^{†*}, Vipul Harsh^{*}, Yajie Zhou[§], Marko Morrison[‡],

B. Aditya Prakash^{†*}, Vyas Sekar^{‡*}, Hui Zhang^{‡*}

[†]Georgia Tech [‡]Carnegie Mellon University [§]University of Maryland ^{*}Conviva

ABSTRACT

Given their world knowledge and reasoning abilities, AI agents hold promise to automate several tasks in systems operations (SysOps). However, LLM responses are inaccurate, hard to verify, and unpredictable, which limits adoption. Some recent works have designed bespoke solutions tackling these challenges for some specific SysOps domains.

In this position paper, we make a case for a constrained creativity paradigm, for developing effective, explainable and efficient agents. The key insight behind the paradigm is that restricting the agent’s actions via a domain-specific language (DSL) with custom operators results in higher verifiability, leading to more effective, explainable and efficient responses. We outline a common abstraction layer that can substantially lower the effort for designing agents with constrained creativity. We demonstrate early promise of this paradigm for two SysOps use cases: Root Cause Analysis and Network Configuration Generation. Our preliminary results show that a constrained creativity agent for RCA achieves 9× higher accuracy than the same LLM prompted directly with the full telemetry, while using 12.5× fewer tokens.

1 INTRODUCTION

Systems operations, or SysOps, is a critical process that keeps networked systems performant, reliable, and secure. This includes various operational tasks; e.g., anomaly detection, incident response and troubleshooting, configuration management, penetration testing, and so on. Classically, SysOps relied on human-driven workflows assisted by software tools and operational playbooks. As we move to sophisticated deployments, rapid design iterations, and greater reliability expectations, such human-timescale workflows are untenable in terms of costs and responsiveness at scale.

LLM-based agents appear promising for SysOps workflows, similar to agentic automation in other disciplines. Such AI-assisted solutions, if realized, can offer several benefits: describing a task in natural language takes less effort than writing manual playbooks [49, 55], and agents write code quickly [45, 46] and provide additional world knowledge for practitioners to use [3]. Agents are being built for anomaly detection [11], RCA of large-scale networked systems [46, 49], red-teaming for vulnerability assessment [7, 40], and network configuration [23, 55].

Compared to other agentic workflows (e.g., customer service chatbots [51], enterprise backend automation [8], coding agents [50]), we argue that agents for SysOps raise the bar in terms of performance and reliability requirements [34]. They need to operate at *scale* and are often on the *critical path* of services, meaning mistakes can have *catastrophic consequences*.¹

To this end, SysOps agents’ design must meet three fundamental requirements, which we call the three Es, as others have also observed [30, 34]:

- *Effectiveness*: The agent should produce reliable, high accuracy results even for new unforeseen scenarios.
- *Efficiency*: The agent should be cost- and time-efficient in responding to tasks.
- *Explainability*: The agent should be able to provide a verifiable and interpretable reasoning or evidence trail that the human practitioner can trust, steer, and hold accountable.

In many diverse SysOps scenarios (§2), naively throwing an LLM or a coding agent at SysOps tasks cannot satisfy one or more of these requirements.

In response, we see several heuristics emerging in academia and industry to tackle these requirements. For instance, recent works in RCA agents explicitly derive some structure from troubleshooting guides, similar to the diagnostic steps a practitioner would take [6, 33]. Red-teaming agents given a vetted vocabulary of high-level attack tactics yielded more reliable results [40]. Researchers design anomaly detection agents to emit structured detection rules that are validated against held-out data before they run in production [11].

Even though these approaches seem disparate at first glance, they actually share a common design philosophy—they *constrain* the agent through *domain-specific structures*. Our primary contribution in this paper is to explicitly define and elevate **constrained creativity** as a high-level design paradigm in designing robust SysOps agents. This may appear paradoxical—to be fully effective on unforeseen scenarios, it appears we need to allow the LLM agent unbounded creativity to express its logic. However, unbounded creativity (e.g., writing arbitrary code) is actually counterproductive to efficiency and explainability in SysOps.² By finding the right

¹Others have called out this agent trilemma between high-stakes, automated, and general purpose [30].

²In many domains of human creativity, constraints are actually good for creativity [21]. Jazz is a famous example where structural constraints serve

domain-specific abstraction for constraining the creativity, we posit that we can achieve efficiency and explainability, without compromising effectiveness.

By explicitly calling out this design paradigm, we hope to create reusable design patterns, best practices, and artifacts that can be used across different settings and realize the full potential of agentic SysOps. We propose a practical realization of constrained creativity via domain-specific languages (DSLs) for different SysOps tasks. Each SysOps task (e.g., RCA, redteaming, config management) has a DSL that encompasses an *abstraction* suitable for the task and *operators* that serve the specific task. In some sense, one can view constrained creativity as a design guideline for harness engineering for SysOps. The grammar of the DSL states how operators compose into a pipeline: which operators may connect and what shapes the wiring may take. The SysOps agent then authors its solutions (e.g., RCA analysis, red teaming tactics) within the confines of the DSL and the operator library. We present qualitative arguments why this specific approach can provide us a basis for achieving the three requirements in §3.

There are several open technical challenges to translate this paradigmatic insight into practice. First, how do we identify a good structure for a given SysOps task? Second, how do we provide best practices for identifying operators? Third, how do we enable continuous evolution to handle new unforeseen scenarios? While we do not yet have prescriptive answers for arbitrary tasks, we show the preliminary qualitative and quantitative promise of answering these questions for two concrete case studies in SysOps: RCA and network configuration. For Root cause analysis, our preliminary proof-of-concept study improves accuracy 9× over a state-of-art baseline while also helping with efficiency and explainability.

2 MOTIVATING USE CASES

We discuss four canonical SysOps tasks. Each predates agentic systems but is increasingly seeing agent adoption or at least exploratory deployments. We highlight how each task imposes the three Es.

Root cause analysis. Modern internet-scale services handle millions of clients across heterogeneous conditions, with complex interactions among clients, network services, third-party services, and backend components. Service providers track KPIs such as page load time, latency, and buffering that affect user experience. When a KPI changes undesirably, an anomaly detection system fires an alert, and an RCA system is tasked with finding its root cause. The RCA system analyzes the available telemetry and returns a rank-ordered set of candidate causes with supporting evidence; e.g., clients

in region B seeing an elevated login failure rate, or a flapping backend microservice whose response time drove up end-to-end latency for the clients assigned to it. An agentic RCA system needs to be **efficient**, since diagnosis has to keep pace with alert volume and with ever-growing telemetry [9, 14, 20, 25]. It needs to be **explainable** and give an evidence trail on what fix is needed and why [9, 14, 20, 25]. And it needs to be **effective**: a wrong diagnosis prolongs the outage [49].

Network red-teaming. To protect networks, security professionals simulate attacker behavior via red team exercises, exposing vulnerabilities, misconfigurations, and chains of vulnerabilities that give an attacker unintended access or control. In an engagement, the agent (or human) is assigned an attack goal and equipped with tools for gathering intel, performing exploits, and controlling infected hosts. A red team agent needs to be **efficient**: an engagement runs within a time limit. It needs to be **explainable**: a finding helps defenders only when they can see which actions the agent took and why, and can reproduce the attack path. And it needs to be **effective**: a missed attack path leaves the vulnerability in production and a false lead wastes practitioner time.

Network configuration generation. In network configuration generation with agents [55], a practitioner describes a desired change in natural language, such as adding a VLAN or updating a BGP policy, and the agent resolves the intent into changes on the relevant configuration paths. The agent needs to be **efficient**: configuration campaigns span large device fleets and must finish inside short maintenance windows. It must be **explainable**: every change ships through change review, and a diff the reviewer cannot trace back to the intent is either rejected, which makes the agent useless, or approved on faith, which makes it dangerous. And it must be **effective**: even one hallucinated key or access-control rule is an outage or a security exposure [56].

Pattern analytics. Internet-scale applications record user behavior; e.g., clicks, searches, page views, and errors. Pattern analytics is the task of finding the behavioral patterns that separate sessions on a KPI: sessions matching the pattern convert, engage, or fail measurably differently from the rest [13, 41]. The steps of a pattern range from raw events to higher-order states derived from them: a buyer’s frustration rising as backend errors accumulate, or booking momentum accelerating at checkout, chained over a session or across weeks of a user’s lifecycle. A pattern analytics system needs to be **efficient**: candidate patterns are combinatorial, each must be matched against millions of sessions, and every session that repeats a harmful pattern is another lost user, so detection has to keep pace with the logs. It needs to be **explainable**: the fix follows from the mechanism, so a finding must show which behavior drives the KPI loss, in which

as a framework for creating innovative outputs [17]. This is also colloquially referred to as the “Green Eggs and Ham” hypothesis [15].

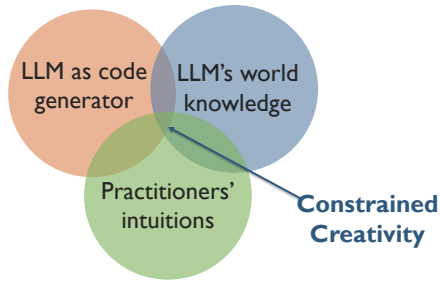


Figure 1: Constrained creativity combines the LLM’s world knowledge and code generation with practitioners’ intuitions.

cohort, and by how much; a correlation alone does not say what to change. And it needs to be **effective**: teams redesign flows and roll back releases on these findings, so a spurious pattern misdirects the change.

Canonical solutions . Conventional solution classes each miss at least one requirement, and the misses are complementary: playbooks run efficiently and explain their answers [32, 42, 44] but are too rigid, classical ML detection and diagnosis pipelines run efficiently but are too narrow [9, 14, 27] or require a human in the loop [12], while LLM-based approaches take on any task but give up efficiency and explainability [3, 7, 28, 46, 49]. As we will see next, there are heuristic efforts appearing to tackle each of these tasks that seem more promising.

3 OVERVIEW OF CONSTRAINED CREATIVITY

We survey academic and industry efforts to create practical agentic workflows for the aforementioned SysOps tasks. We identify a common design philosophy across these seemingly disparate efforts. We systematize this philosophy via the constrained creativity paradigm, and discuss key technical challenges of translating this paradigm into practice.

3.1 Constrained creativity: An emergent idea

At a high level, given a SysOps task, a human or AI agent alternates between reasoning about what to do next and executing one or more actions [53]. The action space and the reasoning logic together determine how effective, efficient, and explainable the work will be.

At one extreme, an expert practitioner’s intuitions are captured in preconfigured tools, playbooks, and workflows. At the other extreme, consider a pure LLM agent running over

a pre-trained foundational model (including data on SysOps-related background knowledge) that can write arbitrary code at machine timescales.

These two extremes entail different tradeoffs in *knowledge* and *action space*. The human approach captures human ingenuity, domain-specific expertise, and explainability, and the tools and playbooks themselves execute efficiently because each is purpose-built and optimized for its task. However, this approach is constrained by the expressivity of the tools and playbooks, the human timescale of authoring new ones, and the practitioner’s own world knowledge. The pure AI approach has a very general action space (raw code) and potentially access to broader knowledge, with strong coding abilities that only improve over time. However, the LLMs used in this approach may have blind spots in specific domains biased by their training, and generally fail to capture the intuition of a human expert. An agent might be more expressive, but can result in unreliable and unexplainable outcomes.

Early success in agents for SysOps. To meet the three E requirements (e.g., [11, 33, 40]), concurrent research efforts have embodied a “middle ground” between these two extremes.

- *Root cause analysis:* While early work on LLM-assisted RCA used an LLM as a blackbox [36, 49], more recent efforts encode the human expert’s diagnosis steps into standard operating procedures that guide the agent [33].
- *Red teaming:* Prior work [7, 40] observed that LLMs directly implementing low-level shell commands have limited effectiveness and efficiency. By making use of specific high-level actions corresponding to typical attack tactics and deterministic services used to track the agent’s observed state of the network, the agent achieved better performance than other LLM-powered attacker baselines.
- *Network management:* To reduce errors, recent work extracts domain-specific invariants from sample queries and encodes them as constraints on the agent: the constraints guide code generation and validate the generated program before it executes [55].
- *Pattern analytics:* Similarly, in user behavioral analytics, rather than asking the LLM to produce interesting insights, recent efforts explicitly prompt agents to encode its hypothesis and validation as dataflow pipelines with pre-defined operators to ensure efficiency and explainability [41].

A common design pattern. While the teams and tasks may be very different, we observe a common theme. For each task, the researchers effectively *constrained the action space* using *domain-specific structures* (e.g., playbooks, dataflow DAG, domain invariants) to achieve the three Es. At the

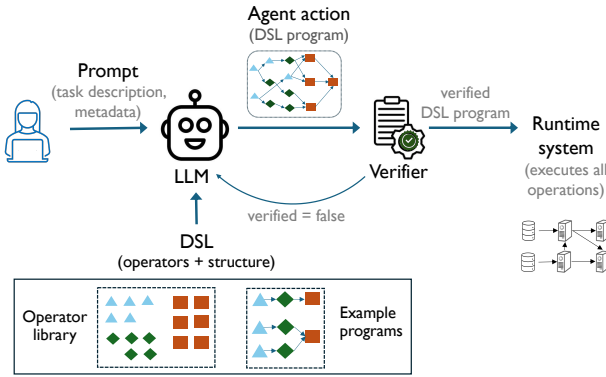


Figure 2: Constrained creativity can be realized via a restricted DSL: a set of operators and the underlying structure that the LLM can compose the operators into.

same time, by leveraging the world knowledge and strong coding abilities of LLMs, all systems could execute tasks with sufficient expressivity to cover most or all scenarios. This is the essence of constrained creativity (Figure 1).

3.2 Formalizing constrained creativity

These disparate efforts show the promise of constraining the scope of an agent’s actions. Our goal is to explicitly elevate and formalize this *constrained creativity* paradigm for building future SysOps agents. By explicitly formalizing the paradigm, we hope to create a reusable set of principles, artifacts, and libraries that researchers and practitioners can benefit from, rather than having to “reinvent the wheel”.

Across these examples, constrained creativity restricts the agent’s action space to predefined operations composed in a particular way. Based on this, we argue that a restricted DSL (domain-specific language) is a natural way to formally capture this insight.³

Restricted DSLs can meet our goals of effectiveness, explainability and efficiency. Restricting the action space reduces the failure surface, making the agent more accurate. The output of the agent can be explained via a relatively simple pipeline of precisely defined operators, as opposed to arbitrary and long programs. Finally, limiting the agent to predefined operators prevents the LLM from performing unbounded actions and improves efficiency.

Expressivity-Verifiability tradeoff. A constrained creativity agent composes its responses from specified operators, like Lego blocks. This costs expressivity as the agent cannot generate any arbitrary response; however the output

is easier to verify which improves overall effectiveness. An agent writing arbitrary code, on the other hand, can produce any output in principle. However, it needs to find the right program from a vast space of possibilities and its output is not easily verifiable. Navigating this trade-off between verifiability and expressivity is the main challenge in the Constrained creativity paradigm.

3.3 Benefits of formalizing the paradigm

A natural question to ask is why formalize constrained creativity as a separate paradigm, realizable via DSLs? The benefits over independent, fragmented solutions mirror the development of DSLs in other contexts historically (as in the case of eBPF [1] or P4 [4]):

- Reusable operator libraries across different domains e.g., Root Cause Analysis in different SysOps contexts
- Custom verification frameworks for enforcing properties in the DSL, rather than checks reinvented for each system
- Agent harnesses customized for agents based on constrained creativity with features for retrying if verification fails, operator development and prompt optimization
- Shared optimized runtime engines with built-in caching and scheduling inherited by all agents
- Development of common visualization and observability tools for debugging and analyzing the agent output

Having the above pieces as parts of a shared abstraction will substantially lower the barrier for developing a new agent for a SysOps task (see Figure 2).

3.4 Choosing the right abstraction

The open question then is in designing the right abstraction, which raises three challenges:

- (1) providing the right structure to the agent to compose these actions (e.g., DAG);
- (2) deciding the set of primitive operators in the DSL that the agent is allowed to compose;
- (3) continuous evolution as the task itself evolves in an open-world setting.

Choosing a suitable structure. Depending on the SysOps task, different structures may be needed to constrain the space of solutions to varying degrees. We outline some possible structures below:

- **Single operator:** the LLM decides one API to call along with its parameters, e.g., “Given a timeseries metric and its description, which anomaly type (spike, long term degradation, changepoint) should be detected?”
- **Chain:** the LLM chains together operators e.g., for alert enhancement: “Alert for Service X at time Y– enrich it for the on-call team: parse the alert → identify owning team → pull relevant log slices → summarize into a page”

³Seen in a historical perspective, this is a comeback for declarative networking [26]. The adoption of declarative networking and DSLs may have faced human user friction in learning a new language/API. Fortunately, LLM-based coding agents have no such complaints!

- **DAG:** a DAG composed of operators. Useful for data analysis tasks such as pattern analytics or root cause analysis e.g., finding root causes for an alert related to some metric: the agent can decide to run parallel comparative analysis against historic time slices or different user slices.
- **Iterated DAG:** a loop whose body is a DAG and whose continuation depends on the results; the LLM decides the next round from what the last returned. “Can an attacker reach the customer database from the public web servers?” An open port leads to credential discovery, which leads to the hosts behind it (red-teaming).
- **Finite state machine:** a finite state machine of operators based on past history. “h1 cannot reach h4; fix the misconfiguration without breaking anything else.” The LLM diagnoses, proposes a change, checks connectivity, and either commits or revisits an earlier step (routing repair).

The above is not an exhaustive list as more complex structures are also possible. In this paper, we do not provide a prescriptive method to determine which structure is suitable beyond our examples; we leave this for future work.

Choosing primitive operators. Practitioners can distill their domain knowledge into the DSL operators. The typing rules and the structure govern how the agent can compose these operators. The most natural way to identify these operators is to collect a representative set of example agent usages and factor out the common pieces. For instance, for RCA, one can factor out meaningful steps for data processing, data cleaning and ML inference (§ 4). As these operators are meant to be composed by an agent (rather than a human) into a DSL program, the library can have a generously large number of operators, not constrained to a handful that humans can reason about at once. Predefined operators also help with efficiency: each operator can be optimized once and reused across queries, and the agent cannot run arbitrary analyses that consume unbounded tokens.

Continuous evolution. Once the DSL (that is, the operators and the structure) is decided, the agent is allowed to choose an action, restricted to this DSL. However, for many tasks such as RCA, the set of use cases evolves over time as the system itself evolves. This can be incorporated by evolving the library of operators over time, which can themselves be designed with the help of coding agents.

3.5 Constrained creativity vs. X

Another natural question is the relationship between constrained creativity and the concurrent suggestions and best practices for building agents. We discuss closely related ideas here and defer readers to §5 for other related topics.

- **Guess-and-check iteration** wraps the LLM in a retry loop: propose, check, retry with feedback [39, 52]. Manual

checking burdens a human with analyzing runtime logs, and automatic checking is challenging in general. Constrained creativity agents are more amenable to checking, so they can use guess-and-check with automatic verification.

- **Prompt and context engineering** change the model’s input: prompt optimizers tune the instructions [2, 22], retrieval adds past incidents to the context [24], and recursive inference lets the model read inputs beyond its context window [54]. They shape what the model reads but do not control the actions it can perform.
- **Tool calling** adds defined tools to the action vocabulary [31, 38]. Each call is checked against its signature, but the pipeline of calls as a whole is not.
- **Guardrails and sandboxes** check each action of the agent, at runtime [35] or in an emulated sandbox before deployment [37]. Neither checks if the composition of actions as a whole is valid.
- **Constrained decoding** masks next-token prediction to follow a given grammar [48]; it can help realize constrained creativity at generation time, emitting programs that conform to the DSL’s syntax.
- **Skills and harness engineering** decide what the agent can call, how calls compose, and what executes. Harness design has grown from prompt templates into runtime and systems design, spanning workflows [19], evaluation, permission control, state management, and review loops [39], with builders making these decisions ad hoc [47]. For SysOps tasks, constrained creativity can serve as the basis for designing a good harness based on its DSL in conjunction with these techniques.

4 ILLUSTRATIVE CASE STUDIES

Root Cause Analysis Case Study. Recall the RCA task of Section 2: given an alert naming the anomalous KPI and its time windows, and optionally an analyst hint, return ranked candidate causes with supporting evidence. The task recurs across data centers, cloud platforms, and internet-scale services, each with its own combination of metrics, events, and logs. We reimagine an RCA agent following constrained creativity.

Choosing the abstraction. We design typed operators based on prior work [12]: hypothesis generation operators propose candidate causes [9, 12–14], hypothesis validation operators test how well each candidate explains the anomaly, and summarization operators surface the final ranked list of root cause candidates.

Sharing the operator library. The operators described above cater to different telemetry data types: timeseries metrics, events, logs, and traces, so the same library is reusable across RCA tasks in other domains with similar telemetry data.

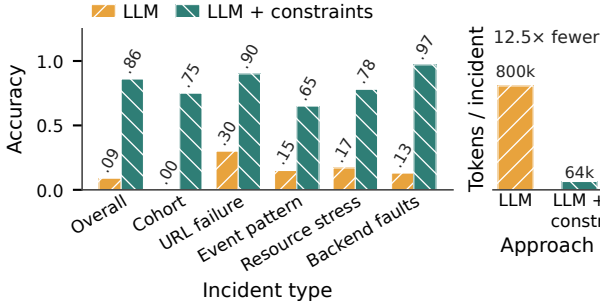


Figure 3: Accuracy overall and by incident type (left) and average tokens per incident (right) for the bare LLM and the same LLM with constraints for RCA.

Verification. Given the incident description and the optional hint, the LLM composes a DAG whose nodes are operator calls with concrete parameters, guided by example playbooks for common incident classes. The verifier ensures that the DAG is semantically correct before handing it over to the runtime engine for execution.

Runtime engine. The runtime engine executes the verified DAG, with built-in caching, parallelism, and scheduling that the agent need not be aware of while composing the DAG. Our prototype implements the engine as a layer on top of Ray [29]. In our cost estimates, the full system costs \$23K per month over a 30 TB monthly telemetry volume, against an estimated \$150K for an agent that freely writes SQL over the same telemetry.

Early results. We built a prototype of this design and compared it against the same LLM (Gemini 3 Pro, auto thinking) prompted directly: the model receives the alert and the full telemetry rendered as text in its prompt and ranks candidate causes in one call. Thinking gives this baseline an internal guess-and-check loop, but the check can only re-read the prompt, never execute over the data. We injected labeled incidents into a simulated internet-scale service built on DeathStarBench [10], spanning client-cohort failures and slowdowns, URL failures, event-sequence patterns, and backend resource, service, and network faults. The constrained prototype places the true root cause among its top five candidates 9× more often than this baseline while consuming 12.5× fewer tokens (Figure 3).

Network Configuration Case Study. We also reimagine the network configuration task mentioned in §2 using the constrained creativity paradigm.

Choosing the abstraction. We define operators to inspect the current network, select the devices and configuration objects, and create, update, or remove entities such as interfaces, VLANs, routing policies, and access-control rules.

Sharing operator library. The same atomic operator families may also recur across WANs, datacenters, and enterprise networks. For example, a WAN may use an operator to update a BGP import policy, while a datacenter may use the same policy-update abstraction for an equal-cost multipath rule. Teams can therefore reuse the core library across domains while adding a small set of local schemas, policies, and device-specific translations.

Verification. Given a configuration update plan from the agent, a verifier applies the proposed change in a sandbox and checks the resulting configuration against schemas and organizational policies. We can use tools such as Batfish to test reachability, isolation, routing behavior, and forwarding changes before deployment [5].

Runtime engine. One can use Ansible [18] to translate the verified plan into concrete device or repository updates, run approved tasks, and record the resulting logs.

5 DISCUSSION

Other advantages. constrained creativity can bring benefits beyond the three Es.

- **Data/model sovereignty.** Constrained creativity enables operators designed to keep data local: the LLM sees only the query and the operator library, never the telemetry. Sensitive data thus never leaves the organization. Furthermore, as other work observed [40], raising the level of abstraction can enable smaller models to work just as well, and those can run locally, helping model sovereignty.
- **Enforceability.** Enforcing a hard operational rule, such as preventing access to data or hosts that are off limits, is difficult with a fully expressive agent: with so many ways to act, it can work around the rule or break it by mistake. Constrained creativity limits the agent to the operators, and when no operator reaches the off-limits data or host, a violation can never happen.
- **Verifiability.** The DSL with its static high-level operators is more amenable to verification than arbitrary code.

Structured design vs. Guess-and-check-iteration. In some sense, constrained creativity exposes a broader debate in how we should be using LLMs and agentic systems. Constrained creativity goes against the trending “AI worldview” of using verification-iteration loops (i.e., guess-and-check), and adopts a “systems philosophy” of correct-by-design systems. This debate was played out in domains like math. Early systems achieved success using structured proofs (e.g., using Lean), but recent wins do not use math-specific scaffolding [16]. Whether math is a special case or SysOps is a different type of task is something that we will need to play out. Guess-and-check iterative loops work when the verification cost is low and the agent has iteration functions. We posit

that SysOps tasks operate in an open-world with high or undefined verification functions. Even if this hypothesis turns out to be false [43], there may be other benefits (efficiency or auxiliary benefits such as those above), where constrained creativity may still be pragmatic.

Beyond SysOps. Several other domains that use agentic systems could potentially benefit by adopting the constrained creativity paradigm. For example, medical and financial data analysis, where the data cannot leave the organization; scientific data processing that requires reproducibility; database tuning where a wrong action is expensive.

REFERENCES

- [1] 2026. eBPF.io. <https://ebpf.io/>
- [2] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.
- [3] Toufique Ahmed, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann, Xuchao Zhang, and Saravan Rajmohan. 2023. Recommending Root-Cause and Mitigation Steps for Cloud Incidents Using Large Language Models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. IEEE.
- [4] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: programming protocol-independent packet processors. *SIGCOMM Comput. Commun. Rev.* 44, 3 (July 2014), 87–95. <https://doi.org/10.1145/2656877.2656890>
- [5] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, and Todd Millstein. 2023. Lessons from the evolution of the Batfish configuration analysis tool. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 122–135.
- [6] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Tianyin Xu. 2024. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. ACM.
- [7] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 847–864. <https://www.usenix.org/conference/usenixsecurity24/presentation/deng>
- [8] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. 2024. WorkArena: How Capable are Web Agents at Solving Common Knowledge Work Tasks?. In *International Conference on Machine Learning (ICML)*.
- [9] Yu Gan, Mingyu Liang, Sundar Dev, David Lo, and Christina Delimitrou. 2021. Sage: Practical and Scalable ML-Driven Performance Debugging in Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM.
- [10] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM.
- [11] Yile Gu, Yifan Xiong, Jonathan Mace, Yuting Jiang, Yigong Hu, Baris Kasikci, and Peng Cheng. 2025. Argos: Agentic Time-Series Anomaly Detection with Autonomous Rule Generation via Large Language Models. arXiv:2501.14170.
- [12] Vipul Harsh, Sayan Sinha, Henry Milner, B. Aditya Prakash, Vyas Sekar, and Hui Zhang. 2026. MoCE: A Mixture-of-Context Aware Experts Framework for Troubleshooting Internet-scale Services. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [13] Vipul Harsh, Sayan Sinha, Henry Milner, Haijie Wu, B. Aditya Prakash, Vyas Sekar, and Hui Zhang. 2025. Automatically Surfacing Opportunities for Improvements in Internet-Scale Applications. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks (HotNets)*. 362–369.
- [14] Vipul Harsh, Wenxuan Zhou, Sachin Ashok, Radhika Niranjana Mysore, Brighten Godfrey, and Sujata Banerjee. 2023. Murphy: Performance Diagnosis of Distributed Cloud Applications. In *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM.
- [15] Catrinel Haught-Tromp. 2017. The Green Eggs and Ham Hypothesis: How Constraints Facilitate Creativity. *Psychology of Aesthetics, Creativity, and the Arts* 11, 1 (2017), 10–17. <https://doi.org/10.1037/aca0000061>
- [16] Chris Hayduk. 2026. The Unreasonable Effectiveness of LLMs in Mathematics. <https://www.chrishayduk.com/p/the-unreasonable-effectiveness-of->
- [17] Carolien Hermans. [n.d.]. On Creativity and Jazz Improvisation – Moving with Constraints. <https://www.conservatoriumvanamsterdam.nl/media/cva/docs/onderzoek/blogs/Blog2CHJazzImprovisation.pdf>.
- [18] Lorin Hochstein and Rene Moser. 2017. *Ansible: Up and Running: Automating configuration management and deployment the easy way*. O'Reilly Media, Inc.
- [19] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*.
- [20] Vimalkumar Jeyakumar, Omid Madani, Ali Parandeh, Ashutosh Kulshreshtha, Weifei Zeng, and Navindra Yadav. 2019. ExplainIt! – A Declarative Root-Cause Analysis Engine for Time Series Data. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*. 333–348.
- [21] Harsh Ketkar and Maria Roche. 2025. Too Much, Too Soon? Early Funding, Technological Unconventionality, and Innovation Capabilities. https://www.hbs.edu/ris/Publication%20Files/25-032_bc4c5e12-45b1-4cd7-9460-7098cb8ad032.pdf. Harvard Business School Working Paper 25-032.
- [22] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations (ICLR)*.
- [23] Patrick T Kon, Jiachen Liu, Yiming Qiu, Weijun Fan, Ting He, Lei Lin, Haoran Zhang, Owen M Park, George S Elengikal, Yuxin Kang, et al.

2024. Iac-eval: A code generation benchmark for cloud infrastructure-as-code programs. *Advances in Neural Information Processing Systems* 37 (2024), 134488–134506.
- [24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [25] Chenghao Liu, Wenzhuo Yang, Himanshu Mittal, Manpreet Singh, Doyen Sahoo, and Steven C. H. Hoi. 2023. PyRCA: A Library for Metric-based Root Cause Analysis. arXiv:2306.11417
- [26] Boon Thau Loo, Tyson Condie, Mimos Garofalakis, David E. Gay, Joseph M. Hellerstein, Petros Maniatis, Raghu Ramakrishnan, Timothy Roscoe, and Ion Stoica. 2009. Declarative networking. *Commun. ACM* 52, 11 (Nov. 2009), 87–95. <https://doi.org/10.1145/1592761.1592785>
- [27] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *Network and Distributed System Security Symposium (NDSS)*.
- [28] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, and George Varghese. 2023. What do LLMs need to Synthesize Correct Router Configurations?. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks (HotNets)*. ACM.
- [29] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elilbol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 561–577. <https://www.usenix.org/conference/osdi18/presentation/moritz>
- [30] Arvind Narayanan. 2026. What will be left for us to work on, ICML Keynote. <https://www.cs.princeton.edu/~arvindn/talks/icml-2026-annotated-slides/>.
- [31] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [32] Vern Paxson. 1999. Bro: A System for Detecting Network Intruders in Real-Time. *Computer Networks* 31, 23–24 (1999), 2435–2463.
- [33] Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, Gaogang Xie, and Dan Pei. 2025. Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis. In *Companion Proceedings of the ACM Web Conference (WWW)*. ACM.
- [34] Stephan Rabanser, Sayash Kapoor, Peter Kirgis, Kangheng Liu, Saiteja Utpala, and Arvind Narayanan. 2026. Towards a Science of AI Agent Reliability. arXiv:2602.16666 [cs.AI] <https://arxiv.org/abs/2602.16666>
- [35] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. arXiv:2310.10501.
- [36] Devjeet Roy, Xuchao Zhang, Rashmi Bhavne, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. Exploring LLM-based Agents for Root Cause Analysis. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*. ACM.
- [37] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations (ICLR)*.
- [38] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [39] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [40] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. 2026. Incalmo: an Autonomous Llm-Assisted System for Red Teaming Multi-Host Networks. In *2026 IEEE Symposium on Security and Privacy (SP)*. 4282–4300. <https://doi.org/10.1109/SP63933.2026.00132>
- [41] Smart Sun, Sayan Sinha, Haijie Wu, Joel Goldfoot, Aditya Ganjam, Jibin Jhan, Evan Chan, Qichu Gong, Vipul Harsh, Bo Lin, Pawas Ranjan, Wei Wang, Zhan Yang, Ningning Hu, B. Aditya Prakash, Vyas Sekar, and Hui Zhang. 2026. Nexa: Automatically Surfacing Business Impacting Insights in E-commerce Applications. In *ACM Conference on AI and Agentic Systems (CAIS), Demo Track*. 1109–1113. <https://doi.org/10.1145/3786335.3813185>
- [42] Yu-Wei Eric Sung, Xiaozheng Tie, Starsky H.Y. Wong, and Hongyi Zeng. 2016. Robotron: Top-down Network Management at Facebook Scale. In *Proceedings of the ACM SIGCOMM 2016 Conference*. ACM.
- [43] Richard Sutton. 2019. The Bitter Lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- [44] The MITRE Corporation. 2024. MITRE Caldera: A Scalable, Automated Adversary Emulation Platform. <https://caldera.mitre.org/>. Accessed: 2026-07-03.
- [45] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. In *International Conference on Machine Learning (ICML)*.
- [46] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2024. RCAGENT: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*. ACM.
- [47] Lilian Weng. 2026. Harness Engineering for Self-Improvement. Lil’Log blog post. <https://lilianweng.github.io/posts/2026-07-04-harness/> Accessed: 2026-07-17.
- [48] Brandon T. Willard and Rémi Louf. 2023. Efficient Guided Generation for Large Language Models. arXiv:2307.09702.
- [49] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?. In *International Conference on Learning Representations (ICLR)*.
- [50] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [51] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations (ICLR)*.
- [52] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [53] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

- [54] Alex L. Zhang, Tim Kraska, and Omar Khattab. 2025. Recursive Language Models. *arXiv:2512.24601*.
- [55] Yajie Zhou, Kevin Hsieh, Sathiya Kumaran Mani, Srikanth Kandula, and Zaoxing Liu. 2025. MeshAgent: Enabling Reliable Network Management with Large Language Models. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 3 (2025), 1–36.
- [56] Yajie Zhou, Jiajun Ruan, Eric S Wang, Sadjad Fouladi, Francis Y Yan, Kevin Hsieh, and Zaoxing Liu. 2026. NetArena: Dynamic Benchmarks for AI Agents in Network Automation. In *The Fourteenth International Conference on Learning Representations*.